



南開大學
Nankai University

计算机学院
语音信息处理实验报告

实验五：说话人识别实验报告

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 6 月 12 日

目录

1 实验一：环境安装与 ECAPA-TDNN 初次推理 (15%)	2
1.1 waveform 读取与数值范围	2
1.2 SpeechBrain checkpoint 结构	3
1.3 embedding 每个维度是否有明确语义?	4
2 实验二：说话人嵌入空间与扰动分析 (35%)	5
2.1 为什么需要对两种 embeddings 联合拟合 t-SNE?	5
2.2 性别和个体的聚类现象	5
2.3 扰动是否可能改变说话人身份?	6
2.4 数据集 domain mismatch	8
3 实验三：说话人验证与说话人辨认 (50%)	8
3.1 Speaker Templates	8
3.2 Dev EER	9
3.3 Identification 和 verification 的区别	10
3.4 为什么说话人验证常用 cosine similarity?	10
3.5 阈值选择与错误代价	11
3.6 为什么不能在 test trials 上调 threshold?	12
3.7 为什么模型能在未见过的 AISHELL speaker 上做 verification/identification 任务? . .	12

1 实验一：环境安装与 ECAPA-TDNN 初次推理（15%）

1.1 waveform 读取与数值范围

查阅 Torchaudio 文档，结合本实验输出，回答：

- `torchaudio.load()` 默认返回的 signal dtype 是什么？
- 默认返回的 signal shape 是什么？`[channel, time]` 还是 `[time, channel]`？
- 使用 SciPy 包的 `scipy.io.wavfile.read()` 方法也可以读取音频。对于 16-bit PCM WAV，`torchaudio.load()` 和 `scipy.io.wavfile.read()` 得到的数据 dtype 和数值范围分别是什么？
- 如果把 `scipy.io.wavfile.read()` 得到的 int16 数据不缩放，直接转成 Tensor 输入 `encode_batch()`，预期会出现什么问题？请用一小段实验验证判断。

答：

`torchaudio.load()` 默认返回的是 `torch.Tensor`。对于常见的 16-bit PCM WAV，默认 `normalize=True`，所以 dtype 通常是 `torch.float32`，数值会被缩放到大约 `[-1.0, 1.0]`。默认 `channels_first=True`，因此 shape 是 `[channel, time]`。

本实验读取示例音频的输出如下：

```
1 signal dtype: torch.float32
2 signal shape: torch.Size([1, 95984])
3 sample rate: 16000
4 min: -0.041259765625
5 max: 0.047088623046875
6 duration sec: 5.999
7
```

而 `scipy.io.wavfile.read()` 读取 16-bit PCM WAV 时不会默认归一化，返回 NumPy 的整数数组。对于 16-bit PCM，理论数值范围为 `[-32768, 32767]`。本实验输出为：

```
1 scipy sample rate: 16000
2 scipy dtype: int16
3 scipy shape: (95984,)
4 scipy min/max: -1352 1543
5
```

因此，二者的关键区别是数值尺度：`torchaudio.load()` 返回 `float32`，约在 `[-1, 1]`；SciPy 返回 `int16`，理论范围为 `[-32768, 32767]`。对本实验文件而言，SciPy 读到的 `[-1352, 1543]` 换算成 torchaudio 默认尺度约为 -0.04126 到 0.04709。

验证代码如下：

```
1 from scipy.io import wavfile
2 import torch
3 import torch.nn.functional as F
4 import numpy as np
5
```

```
6 example_wav = "./data/sample.wav"
7 device = "cuda" if torch.cuda.is_available() else "cpu"
8
9 rate, data = wavfile.read(example_wav)
10
11 wav_bad = torch.from_numpy(data).unsqueeze(0).to(device)
12 wav_good = torch.from_numpy(data.astype(np.float32) / 32768.0).unsqueeze(0).to(device)
13
14 with torch.no_grad():
15     emb_bad = classifier.encode_batch(wav_bad).squeeze().cpu()
16     emb_good = classifier.encode_batch(wav_good).squeeze().cpu()
17
18 emb_bad_l2 = F.normalize(emb_bad, dim=0)
19 emb_good_l2 = F.normalize(emb_good, dim=0)
20
21 print("sample rate:", rate)
22 print("scipy dtype:", data.dtype)
23 print("scipy shape:", data.shape)
24 print("bad waveform range:", wav_bad.min().item(), wav_bad.max().item())
25 print("good waveform range:", wav_good.min().item(), wav_good.max().item())
26 print("bad embedding norm:", emb_bad.norm().item())
27 print("good embedding norm:", emb_good.norm().item())
28 print("cosine similarity:", F.cosine_similarity(emb_bad_l2, emb_good_l2, dim=0).item())
29
```

运行输出为：

```
1 sample rate: 16000
2 scipy dtype: int16
3 scipy shape: (95984,)
4 bad waveform range: -1352 1543
5 good waveform range: -0.041259765625 0.047088623046875
6 bad embedding norm: 234.67430114746094
7 good embedding norm: 234.6707305908203
8 cosine similarity: 0.9999998807907104
9
```

可以看到，直接输入 int16 不一定报错，因为 SpeechBrain 的 `encode_batch()` 内部会将 waveform 转为 float。但“不报错”不等于输入规范。未缩放 int16 的波形尺度和模型期望的 $[-1, 1]$ 浮点 waveform 不一致。这个 checkpoint 中 Fbank、log-mel 和句级均值归一化会抵消一部分整体幅度缩放，所以本例中 embedding 几乎不变；但实际系统中仍应使用与训练和推理接口一致的 waveform 尺度。

1.2 SpeechBrain checkpoint 结构

查阅 SpeechBrain 文档和本地 `ckpt/spkrec-ecapa-cnceleb/hyperparams.yaml`，回答：

- 本 checkpoint 中 modules 包含哪些模块？
- `compute_features`、`mean_var_norm`、`embedding_model`、`classifier` 分别有什么用途？

- `lin_neurons=192` 表示什么？
- `out_n_neurons=2793` 表示什么？
- `encode_batch()` 是否会使用 `classifier`？如果不会，`classifier` 在训练和 `classify_batch()` 中的作用是什么？

答：

本 checkpoint 的 `hyperparams.yaml` 中，`modules` 包含四个模块：

```
1 modules:
2     compute_features
3     mean_var_norm
4     embedding_model
5     classifier
6
```

其中，`compute_features` 使用 `speechbrain.lobes.features.Fbank`，从原始 waveform 中提取 Fbank 声学特征。本实验配置中 `n_mels=80`，表示每一帧提取 80 维 Mel filterbank 特征。

`mean_var_norm` 使用 `speechbrain.processing.features.InputNormalization` 做输入特征归一化。配置为 `norm_type: sentence`、`std_norm: False`，表示按单条语音做句级均值归一化，但不做标准差归一化。它可以减小不同录音音量、信道和整体能量偏移对模型的影响。

`embedding_model` 使用 `speechbrain.lobes.models.ECAPA_TDNN.ECAPA_TDNN`，是主干说话人表征网络。它接收 Fbank 特征并输出 speaker embedding，是后续说话人验证、辨认和相似度计算的核心特征。

`classifier` 使用 `speechbrain.lobes.models.ECAPA_TDNN.Classifier`，是训练阶段接在 embedding 后面的说话人分类头。它把 embedding 映射到训练集中的说话人类别，用于监督训练。

`lin_neurons=192` 表示 ECAPA-TDNN 输出的 embedding 维度是 192。也就是说，每条语音最终会被编码成一个 192 维向量，例如 [192] 或 batch 形式 [1, 1, 192]。`out_n_neurons=2793` 表示训练这个 checkpoint 时分类头输出类别数为 2793，即训练集中有 2793 个说话人类别。

`encode_batch()` 不会使用 `classifier`。根据 SpeechBrain 的 `EncoderClassifier.encode_batch()` 流程，它只执行 `compute_features`、`mean_var_norm`、`embedding_model`，然后直接返回 embedding。`classifier` 的作用主要在训练阶段和 `classify_batch()` 中：训练时，它把 embedding 映射到训练说话人类别并计算分类损失；`classify_batch()` 推理时，它先调用 `encode_batch()` 得到 embedding，再执行 `self.mods.classifier(emb)` 得到训练类别上的后验分数。因此，本实验做说话人比对时主要使用 `encode_batch()` 的 embedding，而不是直接依赖 `classifier`。

1.3 embedding 每个维度是否有明确语义？

embeddings 的 shape 是 [1, 1, 192] 或 [192]。请解释：

- 这 192 个 dimension 是否分别代表“性别”“音高”“口音”“年龄”等人工可解释属性？
- 如果不是，为什么我们仍然可以用这个向量做说话人比对？
- 如果某些维度和性别、口音高度相关，这对系统公平性和隐私有什么潜在影响？

答：

这 192 个 embedding 维度通常没有人工规定的明确语义，不能简单认为第 1 维代表“性别”、第 2 维代表“音高”、第 3 维代表“口音”、第 4 维代表“年龄”。这些维度是神经网络在训练过程中自动学习出来的分布式表示，每一维本身不一定可解释，真正有意义的是整个 192 维向量在空间中的位置和方向。

虽然单个维度不对应明确的人工属性，但 embedding 仍然可以用于说话人比对。训练时模型被要求区分不同说话人，因此会把同一说话人的不同语音映射到相近位置，把不同说话人的语音映射到相对远的位置。实际比对时通常计算两个 embedding 的余弦相似度；如果相似度高，模型认为它们具有相似的说话人特征；如果相似度低，则更可能来自不同说话人。

embedding 中可能隐含编码了性别、年龄、口音、地域、录音设备等信息。即使模型训练目标只是区分说话人，这些属性也可能因为和说话人身份高度相关而被模型利用。这会带来公平性问题：如果训练数据中某些性别、口音、年龄段或地区的人样本不足，模型在这些群体上的错误率可能更高。也会带来隐私问题：embedding 虽然不是原始语音，但攻击者可能训练额外分类器，从 embedding 中推断说话人的性别、年龄、口音、健康状态等敏感属性。因此 speaker embedding 应视为敏感生物特征数据，需要考虑公平性评估、访问控制、加密存储和隐私保护。

2 实验二：说话人嵌入空间与扰动分析（35%）

2.1 为什么需要对两种 embeddings 联合拟合 t-SNE？

本实验同时可视化 clean embeddings 和 perturbed/noisy embeddings。

- 如果分别对 clean embeddings 和 noisy embeddings 各自运行一次 t-SNE，然后直接比较两张图中点的位置，这种比较是否严谨？为什么？

答：

这种比较不严谨。t-SNE 不是一个固定坐标变换，而是针对当前输入数据重新优化一个二维布局。每次单独拟合时，二维坐标轴的方向、旋转、尺度、平移，以及局部簇之间的相对摆放方式都可能不同。即使使用相同的 `random_state`，clean 数据和 noisy 数据的输入集合不同，优化目标也不同，因此两张图里的同一个坐标位置没有一一对应的物理含义。

如果要观察扰动前后的 embedding 是否移动，应该把 clean embeddings 和 perturbed embeddings 拼接在一起，联合运行同一次 t-SNE。这样二者被放在同一个二维投影坐标系中，至少可以定性观察 clean 点和对应扰动点之间的相对位置关系。但联合 t-SNE 仍然主要适合观察局部邻近关系和聚类趋势，不能把二维图中的全局距离、簇间距离和坐标轴方向直接解释为原始 192 维空间中的真实几何距离。定量比较仍应使用原始 embedding 空间中的 cosine similarity 或 embedding drift。

2.2 性别和个体的聚类现象

本实验中，t-SNE 图使用 marker shape 表示 gender，使用 color 表示 speaker id。请结合可视化结果回答：

- 如果观察到男声和女声在 t-SNE 空间中形成相对分离的两个大簇，这说明 speaker embedding 中可能包含哪些信息？
- 这种性别相关的聚类现象对说话人识别系统可能产生哪些影响？

答：

如果观察到男声和女声在 t-SNE 空间中形成相对分离的两个大簇，说明 speaker embedding 中不仅包含说话人身份信息，也可能包含和性别相关的声学信息。例如基频范围、共振峰分布、声道长度、音色、发声方式等因素都可能被模型编码进 embedding。虽然模型训练目标是区分说话人，但性别与声学差异、说话人身份之间存在相关性，所以模型可能把这类信息也作为区分说话人的线索。

从识别准确率角度看，性别信息可能帮助模型做粗粒度区分。例如男声和女声通常更容易被区分，因此性别相关特征可能降低不同性别说话人之间的混淆。但这也可能带来过度依赖非目标属性的问题：说话人识别真正要判断的是“是否同一个人”，而不是“是否同一性别”。如果模型过度依赖性别、口音、年龄等属性，在同一性别内部的细粒度区分、跨域数据或非典型声音样本上可能表现不稳定。

从算法公平性角度看，不同性别群体的识别性能可能存在差异。如果训练数据中男女比例、录音质量、年龄分布或说话风格不均衡，模型可能对某一群体的误拒率或误受率更高。这种差异在门禁、身份核验等应用中会造成不公平的用户体验，甚至带来系统性歧视风险。从隐私保护角度看，如果 embedding 中包含性别等敏感属性，攻击者可能训练额外分类器从 embedding 中推断说话人的性别、年龄、口音、地域甚至健康状态。因此 embedding 应当被视为敏感生物特征数据。

2.3 扰动是否可能改变说话人身份？

请尝试自己实现一种音频扰动方式。完成后，比较扰动前后模型提取到的 speaker embeddings，并分析扰动是否主要保持说话人身份、是否明显改变 embedding 的位置，以及为什么会带来当前观察到的影响量级。

答：

本实验自行实现的扰动为限幅失真 clipping distortion。它模拟麦克风输入过大或录音设备过载时产生的非线性削波：先将 waveform 放大，再把超过阈值的部分裁剪掉，最后重新归一化到合理范围内。

代码如下：

```
1 def clip_distortion(wav, gain=6.0, clip_value=0.35):
2     clipped = torch.clamp(wav * gain, -clip_value, clip_value)
3     clipped = clipped / (clipped.abs().max() + 1e-8)
4     return clipped
5
6 @torch.no_grad()
7 def extract_embedding_from_wav(wav):
8     wavs = wav.squeeze(0).unsqueeze(0).to(device)
9     emb = classifier.encode_batch(wavs).squeeze().detach().cpu()
10    emb = F.normalize(emb, dim=0)
11    return emb.numpy()
12
13 N_EVAL = min(80, len(utts))
14 clip_rows = []
15 clip_embs = []
16
17 for i, row in tqdm(utts.head(N_EVAL).reset_index(drop=True).iterrows(), total=N_EVAL):
18     wav_path = DATA_DIR / row["path"]
19     wav, sr = torchaudio.load(wav_path)
20
```

```

21 wav_clip = clip_distortion(wav, gain=6.0, clip_value=0.35)
22 clip_emb = extract_embedding_from_wav(wav_clip)
23 clean_emb = emb_arr[i]
24
25 cos = float(np.dot(clean_emb, clip_emb) / (
26     np.linalg.norm(clean_emb) * np.linalg.norm(clip_emb) + 1e-8
27 ))
28
29 clip_embs.append(clip_emb)
30 clip_rows.append({
31     "utt_id": row["utt_id"],
32     "spk_id": row["spk_id"],
33     "condition": "clip_gain6_thr035",
34     "cos_clean_clip": cos,
35     "embedding_drift": 1 - cos,
36 })
37
38 clip_drift_df = pd.DataFrame(clip_rows)
39 display(clip_drift_df[["cos_clean_clip", "embedding_drift"]].describe())
40 print("mean cosine:", clip_drift_df["cos_clean_clip"].mean())
41 print("median cosine:", clip_drift_df["cos_clean_clip"].median())
42 print("mean drift:", clip_drift_df["embedding_drift"].mean())
43 print("median drift:", clip_drift_df["embedding_drift"].median())
44

```

输出如下：

```

1      cos_clean_clip  embedding_drift
2 count      80.000000      80.000000
3 mean        0.855752      0.144248
4 std         0.135154      0.135154
5 min         0.604412      0.000051
6 25%         0.719063      0.007646
7 50%         0.892239      0.107761
8 75%         0.992354      0.280937
9 max         0.999949      0.395588
10 mean cosine: 0.8557523235678672
11 median cosine: 0.8922394812107086
12 mean drift: 0.1442476764321327
13 median drift: 0.10776051878929138
14

```

限幅失真主要改变的是录音通道和波形形状，而不是说话人的真实身份。实验中 mean cosine 约为 0.856，median cosine 约为 0.892，说明 clipping 后的语音仍然保留较多说话人身份线索。与此同时，mean drift 约为 0.144，说明限幅失真确实改变了 embedding 的位置，但改变幅度小于前面已有的 noise_snr10 和 channel_8k。前面实验中，noise_snr10 的平均 drift 约为 0.295，channel_8k 的平均 drift 约为 0.486。

原因是 clipping 会引入非线性失真，改变语音的谐波结构、局部能量分布和频谱形状，因此 em-

bedding 不会完全不变。但该扰动没有加入新的背景声，也没有像 8k 通道模拟那样明显删除高频信息，所以对说话人身份线索的破坏相对较轻。

2.4 数据集 domain mismatch

CN-Celeb 和 AISHELL 都是中文语音，是否意味着不存在 domain mismatch？如果将测试语音换成手机近场、会议远场或噪声环境下的语音，speaker embedding 的分布可能发生怎样的变化？

答：

CN-Celeb 和 AISHELL 都是中文语音，但这并不意味着不存在 domain mismatch。语言相同只能说明语种接近，并不能保证录音场景、说话风格、信道条件、背景噪声、说话人构成和任务目标一致。

CN-Celeb 面向说话人识别，包含更真实、更复杂的多体裁场景；AISHELL-1 主要是普通话语音识别语料，本实验使用的 AISHELL mini 数据整体更接近安静、规范的朗读语音，通道和噪声条件更受控。因此，AISHELL 上的实验结果可能偏乐观：安静朗读语音中，背景噪声少、语速和发音较规范、录音质量较稳定，同一说话人的 embedding 更容易聚在一起，不同说话人的 embedding 也更容易分开。

如果测试语音换成手机近场，embedding 可能受到手机麦克风频响、压缩编码、自动增益控制和环境噪声影响；如果换成会议远场，混响、低信噪比、多人干扰和房间声学差异会使同一说话人的 embedding 明显分散，不同说话人的簇也可能互相靠近甚至重叠；如果换成强噪声环境，embedding 可能沿噪声类型或通道条件发生偏移。结合本实验观察，noise_snr10 的平均 drift 约为 0.295，8k 通道模拟的平均 drift 约为 0.486，说明非身份因素可以显著改变 speaker embedding 的位置。

3 实验三：说话人验证与说话人辨认（50%）

3.1 Speaker Templates

如果每个 speaker 只有一条 enrollment，template 是否仍然成立？多条 enrollment 平均的优势是什么？

答：

构建 enrollment templates 的代码如下：

```
1 speaker_templates = {}
2
3 for spk_id, group in enroll_df.groupby("spk_id"):
4     embs = []
5
6     for _, row in group.iterrows():
7         emb = get_embedding_by_path(row["path"])
8         embs.append(emb)
9
10    embs = torch.stack(embs, dim=0)
11    template = F.normalize(embs.mean(dim=0), dim=0)
12
13    speaker_templates[spk_id] = template
14
```

```
15 print("num templates:", len(speaker_templates))
16 print("template dim:", next(iter(speaker_templates.values())).shape)
17
```

如果每个 speaker 只有一条 enrollment, template 仍然成立。根据 template 的定义, speaker template 是该说话人所有 enrollment embeddings 的平均向量再做 L2 normalization。当 $K=1$ 时, 平均向量就是这一条 enrollment 的 embedding 本身。如果这条 embedding 已经做过 L2 normalization, 那么再次归一化后基本不变。因此单条 enrollment 也可以构成 template, 只是代表性较弱。

多条 enrollment 平均的优势是可以得到更稳定、更可靠的说话人表示。单条语音的 embedding 可能受到文本内容、语速、情绪、音量、背景噪声、录音通道或短时发音状态的影响; 多条语音求平均可以抵消一部分偶然因素, 让 template 更接近该说话人的中心位置。平均后再次做 L2 normalization 也很重要, 因为后续 cosine score 假设 template 和 test embedding 都在单位球面上。

3.2 Dev EER

答 :

在 dev trials 上扫描阈值的关键输出如下:

```
1 Dev tau: 0.43569484238441175
2 Dev FAR: 0.0
3 Dev FRR: 0.0
4 Dev EER: 0.0
5
```

在当前数据量级下, 得到 $\text{Dev EER} = 0.0$ 是合理的, 但不能理解为模型已经完全没有错误。本实验的 dev trials 数量较小, 说话人数量也有限, 而且语音主要来自 AISHELL mini 中较干净、较规范的朗读语音。正样本和负样本的 cosine score 在这个小 dev set 上刚好分得比较开时, 就可能存在一个 threshold 使 $\text{FAR}=0$ 、 $\text{FRR}=0$, 因此近似 EER 等于 0。

这种结果更多说明: 在当前实验协议、小规模 dev trials 和 clean 条件下, ECAPA-TDNN 提取的 embeddings 区分度较好; 但它不等价于真实场景中的 EER 就是 0。因为 dev set 太小, 采样到的 impostor trials 可能还不够困难, 没有覆盖足够多相似声音、相似性别、相似口音、噪声、通道变化和远场录音等复杂情况。

如果数据量级大一些, 例如使用全量 AISHELL-1, 仍然出现 $\text{Dev EER} = 0.0$ 就需要更谨慎地看待。全量数据包含更多说话人、更多语音和更多正负 trials, 负样本中更可能出现声音相似的说话人, 正样本中也更可能出现状态变化较大的语音。此时 score 分布通常不会像小样本实验中这样完全分离, 因此 $\text{EER}=0.0$ 的概率会降低。AISHELL-1 仍主要是较安静、规范的普通话朗读语音, 所以 EER 仍可能较低; 但在更大、更复杂、更接近真实使用环境的数据集上, 如果仍报告 0 EER , 则需要检查 enrollment/test 是否泄漏、同一语音是否重复出现、threshold 是否在 test set 上调过、trial 是否过于简单等。

在 test trials 上使用 dev 阈值的输出为:

```
1 {'threshold': np.float64(0.43569484238441175),
2  'FAR': np.float64(0.041666666666666666),
3  'FRR': np.float64(0.0),
4  'accuracy': np.float64(0.9791666666666666)}
5
```

这说明 dev 阈值迁移到 test 后仍有少量 false accept，进一步说明 dev EER=0 不代表真实泛化错误也为 0。

3.3 Identification 和 verification 的区别

请用自己的话说明：

- Speaker identification 关注的是什么问题？
- Speaker verification 关注的是什么问题？
- 为什么二者都可以使用 speaker embedding，但评测协议不同？

答：

Speaker identification 关注的是“这段语音是谁说的”。系统拿一条测试语音的 embedding，与所有已注册说话人的 speaker templates 逐一计算相似度，然后选择分数最高的说话人作为预测结果。在 closed-set identification 中，系统假设测试语音一定来自已注册说话人；在 open-set identification 中，如果最高分低于阈值，还可以输出 unknown。

Speaker verification 关注的是“这段语音是不是某个指定的人说的”。它的输入不是单独一条测试语音，而是一个 enrollment speaker 和一条 test utterance 组成的 trial。系统只需要计算该测试语音和指定 speaker template 的相似度，再根据 threshold 判断 same speaker 或 different speaker。

二者都可以使用 speaker embedding，因为 embedding_model 学到的是一个固定维度的说话人表征空间：同一说话人的 embeddings 应该更接近，不同说话人的 embeddings 应该更远。因此，无论是“一对一”的 verification，还是“一对多”的 identification，都可以基于 embedding 之间的 cosine similarity 完成。

但二者的评测协议不同。Verification 的每个 trial 是 `enroll_spk_id + test_utt_id + label`，输出是 same/different，常用 FAR、FRR、EER、accuracy 等指标；identification 的每个 trial 通常是一条 test utterance，系统需要在所有 enrolled speakers 中选出一个 speaker id，closed-set 常看 top-1 accuracy，open-set 还要考虑 unknown reject 的阈值和拒识性能。本 notebook 当前 closed-set identification accuracy 为 1.0，而 verification 在 dev 上 EER 为 0.0、在 test threshold 下 FAR 约 0.0417、FRR 为 0.0，说明这两个任务虽然共用 embedding，但决策形式和错误类型并不相同。

3.4 为什么说话人验证常用 cosine similarity？

说话人验证为什么常用 cosine similarity 比较两个 embeddings？请从以下角度回答：

- embedding 是固定维度向量；
- L2 normalization 后 dot product 与 cosine similarity 的关系；
- 模型训练中常用 angular / margin-based 分类目标；
- cosine score 是否等价于概率？

答：

说话人验证常用 cosine similarity，是因为 speaker embedding 是固定维度向量。本实验中 ECAPA-TDNN 输出的是 192 维 embedding。把不同长度的语音都编码成固定长度向量后，就可以直接比较两个向量在 embedding 空间中的相似程度。

在本实验代码中，提取 embedding 后会做 L2 normalization，speaker template 也会在平均后再做 L2 normalization。对于两个已经归一化到单位长度的向量 e 和 z ，有：

$$\cos(e, z) = \frac{e^T z}{\|e\| \|z\|} = e^T z$$

也就是说，L2 normalization 之后，dot product 和 cosine similarity 等价。因此代码里既可以显式计算 cosine，也可以在归一化后用点积作为 cosine score。

从训练目标看，现代 speaker embedding 模型常使用 angular 或 margin-based 分类目标，例如 AM-Softmax、AAM-Softmax、ArcFace 类目标。这类目标强调 embedding 的方向或角度关系，鼓励同一说话人的 embeddings 在角度空间中更紧凑，不同说话人的 embeddings 在角度空间中分得更开。因此推理时使用 cosine similarity 比较 embedding 方向，与这类训练目标的几何假设比较一致。

但 cosine score 不等价于概率。score 高只能说明两个 embeddings 在当前模型空间中更相似，不能直接解释为“这两段语音属于同一个人的概率是 score”。例如 cosine=0.8 不是 80% 的概率。是否判为 same speaker 还取决于 threshold，而 threshold 应根据 dev set、应用场景和错误代价来选择。如果需要概率意义，还需要额外的 score calibration。

3.5 阈值选择与错误代价

假设一个系统用于门禁，另一个系统用于课堂签到：

- 哪个系统更不能接受 false accept？
- 哪个系统更不能接受 false reject？
- 这两个系统是否应该使用同一个 threshold？

答：

门禁系统更不能接受 false accept。False accept 表示系统把冒充者或非授权人员错误接受为合法用户，这会直接造成安全风险。因此门禁场景通常倾向于使用更高的 threshold，让系统更保守，只有相似度足够高才接受。这样可以降低 FAR，但代价是 FRR 可能升高，也就是合法用户偶尔被拒绝，需要重试或改用其他验证方式。

课堂签到系统相对更不能接受 false reject。False reject 表示真实到课的学生被系统错误拒绝，可能导致被误记为缺勤，影响考勤公平性。课堂签到当然也不希望 false accept，因为这可能允许代签或冒签；但和门禁相比，它通常不是高安全风险场景，错误代价结构不同。因此课堂签到可能会选择相对低一些的 threshold，减少真实学生被拒绝的情况，同时再结合人工复核、位置、时间窗口或二次确认降低冒签风险。

这两个系统不应该直接使用同一个 threshold。Threshold 本质上是在 FAR 和 FRR 之间做权衡：提高 threshold 通常降低 false accept、提高 false reject；降低 threshold 通常降低 false reject、提高 false accept。门禁和课堂签到的错误代价不同，所以应根据各自应用场景在 dev set 上单独选择 threshold，而不是只使用 EER 点或共用一个固定阈值。

3.6 为什么不能在 test trials 上调 threshold?

- 请解释 dev set 和 test set 的区别。
- 如果你先在 test trials 上扫描 threshold，再报告该 threshold 下的 test performance，这样做有什么问题？

答：

Dev set 和 test set 的作用不同。Dev set 用于开发阶段，包括选择 threshold、比较模型、调参数、决定后处理策略等。Test set 应该只在最终评估时使用，用来模拟模型面对未见数据时的真实性能。

如果先在 test trials 上扫描 threshold，再报告该 threshold 下的 test performance，就相当于把 test set 信息用于模型选择或决策规则选择。这会造成数据泄漏，使 test set 不再是独立评估集。得到的 performance 会偏乐观，因为 threshold 已经针对这批 test trials 的分数分布和标签被优化过。

这种做法的问题是泛化能力被高估。比如在 test set 上扫描得到的 threshold 可能刚好避开了这批 test trials 的错误，但换一批真实数据后未必有效。正确流程应该是：在 dev trials 上扫描 threshold，固定 tau；然后只把这个 tau 应用到 test trials 上，报告 test FAR、FRR、accuracy 或 EER 等指标。本 notebook 就是先在 dev 上得到 tau，再在 test trials 上应用该 tau，这样 test 结果更能反映泛化性能。

3.7 为什么模型能在未见过的 AISHELL speaker 上做 verification/identification 任务？

CN-Celeb checkpoint 的训练目标本质上是对训练说话人做分类，out_n_neurons=2793。但实验中我们拿它的 embedding 去验证/辨认 AISHELL 中未见过的说话人。请解释：

- 为什么分类训练得到的中间 embedding 可以迁移到未见过的 speaker？
- classifier head 和 embedding_model 在用途上有什么区别？
- 如果只使用 classifier 的输出类别，能否识别 AISHELL speaker？

答：

CN-Celeb checkpoint 训练时虽然本质上是在做 2793 个训练说话人的分类，但分类训练的中间层 embedding_model 学到的不是简单的“训练集 speaker id 查表”，而是能区分说话人声学特征的代表空间。为了正确分类训练说话人，模型必须学习与说话人身份相关的普遍线索，例如声道特征、音色、发声习惯、频谱结构等。这些线索对未见过的新说话人也有用，因此中间 embedding 可以迁移到 AISHELL speaker 上。

embedding_model 和 classifier head 的用途不同。embedding_model 负责把任意输入语音编码成一个固定维度 speaker embedding，它更像一个通用的说话人特征提取器；classifier head 则把 embedding 映射到训练时定义好的 2793 个 CN-Celeb 说话人类别，用于训练阶段计算分类损失，或在 classify_batch() 中输出训练类别上的后验分数。

在本实验中，我们使用的是 embedding_model 的输出，而不是 classifier 的输出类别。对于 AISHELL 中未见过的 speaker，虽然模型没有这些人的类别标签，但它仍然可以把他们的语音映射到 embedding 空间中。之后通过 enrollment templates、cosine similarity 和 threshold 完成 verification/identification。

如果只使用 classifier 的输出类别，不能真正识别 AISHELL speaker。因为 classifier 的 `out_n_neurons=2793` 对应的是 CN-Celeb 训练说话人，而不是 AISHELL 的 speaker id。对 AISHELL 语音调用 classifier，最多只能得到“最像哪个 CN-Celeb 训练说话人”的输出，这和识别 AISHELL speaker 没有直接对应关系。